
latest

Search docs

User Documentation

- Introduction
- Installation
- Algorithms
- Running Experiments
- Experiment Outputs
- Plotting Results

Introduction to RL

- Part 1: Key Concepts in RL
- Part 2: Kinds of RL Algorithms
- Part 3: Intro to Policy Optimization

Resources

- Spinning Up as a Deep RL Researcher
- Key Papers in Deep RL
- Exercises
- Benchmarks for Spinning Up Implementations

Algorithms Docs

- Vanilla Policy Gradient
- Trust Region Policy Optimization
- Proximal Policy Optimization
  - Background
    - Quick Facts
    - Key Equations
    - Exploration vs. Exploitation
    - Pseudocode
  - Documentation
    - Documentation: PyTorch Version
    - Saved Model Contents: PyTorch Version
    - Documentation: Tensorflow Version
    - Saved Model Contents: Tensorflow Version
  - References
    - Relevant Papers
    - Why These Papers?
    - Other Public Implementations
- Deep Deterministic Policy Gradient
- Twin Delayed DDPG
- Soft Actor-Critic

Utilities Docs

- Logger
- Plotter
- MPI Tools
- Run Utils

Etc.

- Acknowledgements
- About the Author

Spinning Up

- Docs »
- Proximal Policy Optimization
- Edit on GitHub

Proximal Policy Optimization¶

Table of Contents

- Proximal Policy Optimization
  - Background
    - Quick Facts
    - Key Equations
    - Exploration vs. Exploitation
    - Pseudocode
  - Documentation
    - Documentation: PyTorch Version
    - Saved Model Contents: PyTorch Version
    - Documentation: Tensorflow Version
    - Saved Model Contents: Tensorflow Version
  - References
    - Relevant Papers
    - Why These Papers?
    - Other Public Implementations

Background¶

(Previously: Background for TRPO)

PPO is motivated by the same question as TRPO: how can we take the biggest possible improvement step on a policy using the data we currently have, without stepping so far that we accidentally cause performance collapse? Where TRPO tries to solve this problem with a complex second-order method, PPO is a family of first-order methods that use a few other tricks to keep new policies close to old. PPO methods are significantly simpler to implement, and empirically seem to perform at least as well as TRPO.

There are two primary variants of PPO: PPO-Penalty and PPO-Clip.

PPO-Penalty approximately solves a KL-constrained update like TRPO, but penalizes the KL-divergence in the objective function instead of making it a hard constraint, and automatically adjusts the penalty coefficient over the course of training so that it’s scaled appropriately.

PPO-Clip doesn’t have a KL-divergence term in the objective and doesn’t have a constraint at all. Instead relies on specialized clipping in the objective function to remove incentives for the new policy to get far from the old policy.

Here, we’ll focus only on PPO-Clip (the primary variant used at OpenAI).

Quick Facts¶

- PPO is an on-policy algorithm.
- PPO can be used for environments with either discrete or continuous action spaces.
- The Spinning Up implementation of PPO supports parallelization with MPI.

Key Equations¶

PPO-clip updates policies via

$$\theta_{k+1} = \arg \max_{\theta} \underset{s,a \sim \pi_{\theta_k}}{{\mathrm E}}\left[L(s,a,\theta_k, \theta)\right],$$

typically taking multiple steps of (usually minibatch) SGD to maximize the objective. Here L is given by

$$L(s,a,\theta_k,\theta) = \min\left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a), \;\; \text{clip}\left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, 1 - \epsilon, 1+\epsilon \right) A^{\pi_{\theta_k}}(s,a) \right),$$

in which ϵ is a (small) hyperparameter which roughly says how far away the new policy is allowed to go from the old.

This is a pretty complex expression, and it's hard to tell at first glance what it's doing, or how it helps keep the new policy close to the old policy. As it turns out, there's a considerably simplified version [1] of this objective which is a bit easier to grapple with (and is also the version we implement in our code):

$$L(s,a,\theta_k,\theta) = \min\left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)} A^{\pi_{\theta_k}}(s,a), \;\; g(\epsilon, A^{\pi_{\theta_k}}(s,a)) \right),$$

where

$$g(\epsilon, A) = \left\{ \begin{array}{ll} (1 + \epsilon) A & A \geq 0 \\ (1 - \epsilon) A & A < 0. \end{array} \right.$$

To figure out what intuition to take away from this, let's look at a single state-action pair (s,a), and think of cases.

Advantage is positive: Suppose the advantage for that state-action pair is positive, in which case its contribution to the objective reduces to

$$L(s,a,\theta_k,\theta) = \min\left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, (1 + \epsilon) \right) A^{\pi_{\theta_k}}(s,a).$$

Because the advantage is positive, the objective will increase if the action becomes more likely—that is, if $\pi_{\theta}(a|s)$ increases. But the min in this term puts a limit to how *much* the objective can increase. Once $\pi_{\theta}(a|s) > (1+\epsilon) \pi_{\theta_k}(a|s)$, the min kicks in and this term hits a ceiling of $(1+\epsilon) A^{\pi_{\theta_k}}(s,a)$. Thus: *the new policy does not benefit by going far away from the old policy*.

Advantage is negative: Suppose the advantage for that state-action pair is negative, in which case its contribution to the objective reduces to

$$L(s,a,\theta_k,\theta) = \max\left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_k}(a|s)}, (1 - \epsilon) \right) A^{\pi_{\theta_k}}(s,a).$$

Because the advantage is negative, the objective will increase if the action becomes less likely—that is, if \pi_{\theta}(a|s) decreases. But the max in this term puts a limit to how *much* the objective can increase. Once \pi_{\theta}(a|s) < (1-\epsilon) \pi_{\theta_k}(a|s), the max kicks in and this term hits a ceiling of (1-\epsilon) A^{\pi_{\theta_k}}(s,a). Thus, again: *the new policy does not benefit by going far away from the old policy*.

What we have seen so far is that clipping serves as a regularizer by removing incentives for the policy to change dramatically, and the hyperparameter \epsilon corresponds to how far away the new policy can go from the old while still profiting the objective.

You Should Know

While this kind of clipping goes a long way towards ensuring reasonable policy updates, it is still possible to end up with a new policy which is too far from the old policy, and there are a bunch of tricks used by different PPO implementations to stave this off. In our implementation here, we use a particularly simple method: early stopping. If the mean KL-divergence of the new policy from the old grows beyond a threshold, we stop taking gradient steps.

When you feel comfortable with the basic math and implementation details, it's worth checking out other implementations to see how they handle this issue!

[1] See this note for a derivation of the simplified form of the PPO-Clip objective.

Exploration vs. Exploitation¶

PPO trains a stochastic policy in an on-policy way. This means that it explores by sampling actions according to the latest version of its stochastic policy. The amount of randomness in action selection depends on both initial conditions and the training procedure. Over the course of training, the policy typically becomes progressively less random, as the update rule encourages it to exploit rewards that it has already found. This may cause the policy to get trapped in local optima.

Pseudocode¶

\begin{algorithm}[H] \caption{PPO-Clip} \label{alg1} \begin{algorithmic}[1] \STATE Input: initial policy parameters θ_0, initial value function parameters ϕ_0 \FOR{$k = 0,1,2,...$} \STATE Collect set of trajectories ${\mathcal D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment. \STATE Compute rewards-to-go $\hat{R}_t$. \STATE Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}. \STATE Update the policy by maximizing the PPO-Clip objective: \begin{equation*} \theta_{k+1} = \arg \max_{\theta} \frac{1}{|{\mathcal D}_k| T} \sum_{\tau \in {\mathcal D}_k} \sum_{t=0}^T \min\left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t,a_t), \;\; g(\epsilon, A^{\pi_{\theta_k}}(s_t,a_t)) \right), \end{equation*} typically via stochastic gradient ascent with Adam. \STATE Fit value function by regression on mean-squared error: \begin{equation*} \phi_{k+1} = \arg \min_{\phi} \frac{1}{|{\mathcal D}_k| T} \sum_{\tau \in {\mathcal D}_k} \sum_{t=0}^T\left(V_{\phi} (s_t) - \hat{R}_t \right)^2, \end{equation*} typically via some gradient descent algorithm. \ENDFOR \end{algorithmic} \end{algorithm}

Documentation¶

You Should Know

In what follows, we give documentation for the PyTorch and Tensorflow implementations of PPO in Spinning Up. They have nearly identical function calls and docstrings, except for details relating to model construction. However, we include both full docstrings for completeness.

Documentation: PyTorch Version¶

`spinup.ppo_pytorch`(*env_fn*, *actor_critic=<MagicMock spec='str' id='140554322637768'>*, *ac_kwargs={}*, *seed=0*, *steps_per_epoch=4000*, *epochs=50*, *gamma=0.99*, *clip_ratio=0.2*, *pi_lr=0.0003*, *vf_lr=0.001*, *train_pi_iters=80*, *train_v_iters=80*, *lam=0.97*, *max_ep_len=1000*, *target_kl=0.01*, *logger_kwargs={}*, *save_freq=10*)¶

Proximal Policy Optimization (by clipping),

with early stopping based on approximate KL

Parameters:

- **env_fn** – A function which creates a copy of the environment. The environment must satisfy the OpenAI Gym API.
- **actor_critic** –

  The constructor method for a PyTorch Module with a `step` method, an `act` method, a `pi` module, and a `v` module. The `step` method should accept a batch of observations and return:

Symbol	Shape	Description
`a`	(batch, act_dim)	Numpy array of actions for each observation.
`v`	(batch,)	Numpy array of value estimates for the provided observations.
`logp_a`	(batch,)	Numpy array of log probs for the actions in `a`.

  The `act` method behaves the same as `step` but only returns `a`.

  The `pi` module's forward call should accept a batch of observations and optionally a batch of actions, and return:

Symbol	Shape	Description
`pi`	N/A	Torch Distribution object, containing a batch of distributions describing the policy for the provided observations.
`logp_a`	(batch,)	Optional (only returned if batch of actions is given). Tensor containing the log probability, according to the policy, of the provided actions. If actions not given, will contain `None`.

  The `v` module's forward call should accept a batch of observations and return:

Symbol	Shape	Description
`v`	(batch,)	Tensor containing the value estimates for the provided observations. (Critical: make sure to flatten this!)

- **ac_kwargs** (*dict*) – Any kwargs appropriate for the ActorCritic object you provided to PPO.

- **seed** (*int*) – Seed for random number generators.
- **steps_per_epoch** (*int*) – Number of steps of interaction (state-action pairs) for the agent and the environment in each epoch.
- **epochs** (*int*) – Number of epochs of interaction (equivalent to number of policy updates) to perform.
- **gamma** (*float*) – Discount factor. (Always between 0 and 1.)
- **clip_ratio** (*float*) – Hyperparameter for clipping in the policy objective. Roughly: how far can the new policy go from the old policy while still profiting (improving the objective function)? The new policy can still go farther than the clip_ratio says, but it doesn't help on the objective anymore. (Usually small, 0.1 to 0.3.) Typically denoted by \epsilon.
- **pi_lr** (*float*) – Learning rate for policy optimizer.
- **vf_lr** (*float*) – Learning rate for value function optimizer.
- **train_pi_iters** (*int*) – Maximum number of gradient descent steps to take on policy loss per epoch. (Early stopping may cause optimizer to take fewer than this.)
- **train_v_iters** (*int*) – Number of gradient descent steps to take on value function per epoch.
- **lam** (*float*) – Lambda for GAE-Lambda. (Always between 0 and 1, close to 1.)
- **max_ep_len** (*int*) – Maximum length of trajectory / episode / rollout.
- **target_kl** (*float*) – Roughly what KL divergence we think is appropriate between new and old policies after an update. This will get used for early stopping. (Usually small, 0.01 or 0.05.)
- **logger_kwargs** (*dict*) – Keyword args for EpochLogger.
- **save_freq** (*int*) – How often (in terms of gap between epochs) to save the current policy and value function.

Saved Model Contents: PyTorch Version¶

The PyTorch saved model can be loaded with `ac = torch.load('path/to/model.pt')`, yielding an actor-critic object (`ac`) that has the properties described in the docstring for `ppo_pytorch`.

You can get actions from this model with

```
actions = ac.act(torch.as_tensor(obs, dtype=torch.float32))
```

Documentation: Tensorflow Version¶

`spinup.ppo_tf1`(*env_fn*, *actor_critic=<function mlp_actor_critic>*, *ac_kwargs={}*, *seed=0*, *steps_per_epoch=4000*, *epochs=50*, *gamma=0.99*, *clip_ratio=0.2*, *pi_lr=0.0003*, *vf_lr=0.001*, *train_pi_iters=80*, *train_v_iters=80*, *lam=0.97*, *max_ep_len=1000*, *target_kl=0.01*, *logger_kwargs={}*, *save_freq=10*)¶

Proximal Policy Optimization (by clipping),

with early stopping based on approximate KL

Parameters:

- **env_fn** – A function which creates a copy of the environment. The environment must satisfy the OpenAI Gym API.
- **actor_critic** –

  A function which takes in placeholder symbols for state, `x_ph`, and action, `a_ph`, and returns the main outputs from the agent's Tensorflow computation graph:

Symbol	Shape	Description
`pi`	(batch, act_dim)	Samples actions from policy given states.
`logp`	(batch,)	Gives log probability, according to the policy, of taking actions `a_ph` in states `x_ph`.
`logp_pi`	(batch,)	Gives log probability, according to the policy, of the action sampled by `pi`.
`v`	(batch,)	Gives the value estimate for states in `x_ph`. (Critical: make sure to flatten this!)

- **ac_kwargs** (*dict*) – Any kwargs appropriate for the actor_critic function you provided to PPO.
- **seed** (*int*) – Seed for random number generators.
- **steps_per_epoch** (*int*) – Number of steps of interaction (state-action pairs) for the agent and the environment in each epoch.
- **epochs** (*int*) – Number of epochs of interaction (equivalent to number of policy updates) to perform.
- **gamma** (*float*) – Discount factor. (Always between 0 and 1.)
- **clip_ratio** (*float*) – Hyperparameter for clipping in the policy objective. Roughly: how far can the new policy go from the old policy while still profiting (improving the objective function)? The new policy can still go farther than the clip_ratio says, but it doesn't help on the objective anymore. (Usually small, 0.1 to 0.3.) Typically denoted by \epsilon.
- **pi_lr** (*float*) – Learning rate for policy optimizer.
- **vf_lr** (*float*) – Learning rate for value function optimizer.
- **train_pi_iters** (*int*) – Maximum number of gradient descent steps to take on policy loss per epoch. (Early stopping may cause optimizer to take fewer than this.)
- **train_v_iters** (*int*) – Number of gradient descent steps to take on value function per epoch.
- **lam** (*float*) – Lambda for GAE-Lambda. (Always between 0 and 1, close to 1.)
- **max_ep_len** (*int*) – Maximum length of trajectory / episode / rollout.
- **target_kl** (*float*) – Roughly what KL divergence we think is appropriate between new and old policies after an update. This will get used for early stopping. (Usually small, 0.01 or 0.05.)
- **logger_kwargs** (*dict*) – Keyword args for EpochLogger.
- **save_freq** (*int*) – How often (in terms of gap between epochs) to save the current policy and value function.

Saved Model Contents: Tensorflow Version¶

The computation graph saved by the logger includes:

Key	Value
`x`	Tensorflow placeholder for state input.
`pi`	Samples an action from the agent, conditioned on states in `x`.
`v`	Gives value estimate for states in `x`.

This saved model can be accessed either by

- running the trained policy with the test_policy.py tool,
- or loading the whole saved graph into a program with restore_tf_graph.

References¶

Relevant Papers¶

- Proximal Policy Optimization Algorithms, Schulman et al. 2017
- High Dimensional Continuous Control Using Generalized Advantage Estimation, Schulman et al. 2016
- Emergence of Locomotion Behaviours in Rich Environments, Heess et al. 2017

Why These Papers?¶

Schulman 2017 is included because it is the original paper describing PPO. Schulman 2016 is included because our implementation of PPO makes use of Generalized Advantage Estimation for computing the policy gradient. Heess 2017 is included because it presents a large-scale empirical analysis of behaviors learned by PPO agents in complex environments (although it uses PPO-penalty instead of PPO-clip).

Other Public Implementations¶

- Baselines
- ModularRL (Caution: this implements PPO-penalty instead of PPO-clip.)
- rllab (Caution: this implements PPO-penalty instead of PPO-clip.)
- rllib (Ray)

Next Previous

© Copyright 2018, OpenAI. Revision `038665d6`.

Built with Sphinx using a theme provided by Read the Docs.